

QiMeng-CodeV-SVA: Training Specialized LLMs for Hardware Assertion Generation via RTL-Grounded Bidirectional Data Synthesis

Yutong Wu

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
wuyutong22s@ict.ac.cn

Chenrui Cao

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China
University of Science and Technology
of China
Hefei, Anhui, China

Pengwei Jin

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China

Di Huang

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China

Rui Zhang

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China

Xishan Zhang

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China
Cambricon Technologies
Beijing, China

Zidong Du

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China

Qi Guo

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China

Xing Hu*

SKL of Processors, Institute of
Computing Technology, CAS
Beijing, China

Abstract

SystemVerilog Assertions (SVAs) are crucial for hardware verification. Recent studies leverage general-purpose LLMs to translate natural language properties to SVAs (NL2SVA), but they perform poorly due to limited data. We propose a data synthesis framework to tackle two challenges: the scarcity of high-quality real-world SVA corpora and the lack of reliable methods to determine NL-SVA semantic equivalence. For the former, large-scale open-source RTLs are used to guide LLMs to generate real-world SVAs; for the latter, bidirectional translation serves as a data selection method. With the synthesized data, we train CodeV-SVA, a series of SVA generation models. Notably, CodeV-SVA-14B achieves 75.8% on NL2SVA-Human and 84.0% on NL2SVA-Machine in Func.@1, matching or exceeding advanced LLMs like GPT-5 and DeepSeek-R1.

1 Introduction

Assertion-based hardware formal verification plays a vital role in the digital hardware design flow [30]. Based on natural-language specifications and Register Transfer Level (RTL) code, verification engineers need to formulate temporal logical assertions, namely SystemVerilog Assertions (SVAs) [20], and employ simulation or formal verification (FV) tools (e.g., Cadence JasperGold) to ensure that the RTL implementation satisfies the specified constraints corresponding to the SVAs.

Given the substantial manual effort and expertise required to craft high-quality SVAs, the automatic generation of SVAs has

emerged as an important research area. Classical rule-based methods [10, 12] mine SVAs from simulation traces, but they depend on the availability of golden RTL designs and are thus hard to apply to real-world verification scenarios. More recent work focuses on employing large language models (LLMs) to analyze specifications and RTL, first formulating verification properties in natural language and then translating those properties into corresponding SVAs [5, 19, 28, 29, 33]. The common weakness of these methods is their usage of general-purpose LLMs (e.g., DeepSeek-V3.1 [7]) for the natural-language-to-SVA (NL2SVA) translation. Due to the lack of background knowledge, general-purpose LLMs like DeepSeek-V3.1 tend to underperform on this specialized translation task [14, 15] (see Table 2). In addition, advanced LLMs such as GPT-5 [3] and DeepSeek-R1 [6] either fail to meet proprietary requirements of hardware companies or impose very high deployment costs. Therefore, developing LLMs that are both accurate for SVA generation and affordable to deploy is essential.

However, training SVA LLMs is hampered by the lack of high-quality training data [21], which stems from two main challenges. **(1) The scarcity of high-quality SVA corpora in the wild.** Publicly available human-written SVAs mostly appear in textbooks and a handful of open-source repositories, but these sources are limited in scale. For example, Hybrid-NL2SVA [32] contains only 4,070 SVAs from textbooks; from the DeepCircuitX [16] dataset, we can only extract 5,638 SVAs in over 4K open-source repositories; By contrast, open datasets of RTL code (e.g., DeepCircuitX [16] and CodeV [37]) cover on the order of 10^5 NL-Verilog pairs, a scale and that far exceed those of SVA data. **(2) The lack of reliable methods to**

*Corresponding author.

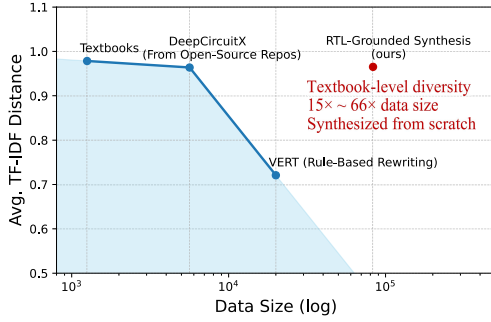


Figure 1: The relationship between data size and average 3-gram TF-IDF distance of different SVA sources.

determine semantic equivalence between NL properties and model-generated SVAs. LLM-based data synthesis pipelines typically rely on post-selection to filter low-quality samples [36], but NL-SVA pairs are hard to validate automatically: Simply using formal verification tools to check the SVA under RTL is insufficient because trivial or vacuous assertions (e.g., `assert property (1'b1)`) will pass against any RTL, yet show no alignment with the NL description [15]; Another approach, LLM-as-a-judge [11], which prompts an LLM to judge whether the NL and the generated SVA are consistent, struggles with NL ambiguity and the subtle syntax of SVA (e.g., operator precedence, see Figure 2).

To tackle these challenges, we begin with two key observations: (1) **LLMs can build upon open-source RTL code as design-under-tests (DUTs) to synthesize a large amount of high-quality SVAs, thereby alleviating data scarcity.** To validate this, we analyze the SVAs from textbooks and open-source repositories, SVAs automatically generated by rule-based rewriting in prior work VERT [21], and RTL-grounded. We measure both the scale and diversity (using the 3-gram TF-IDF distance [26] as the metric) of them. In Figure 1, as data size grows, the diversity of the rule-based method drops significantly, while the RTL-grounded approach maintains diversity close to that of open-source repositories, demonstrating its stronger data augmentation capability. (2) **Both NL2SVA and SVA2NL translation incur information loss. Consequently, if an SVA, after being converted to NL by LLMs and then back to SVA (bidirectional translation), remains logically equivalent to the original one, it is highly probable that no information loss has occurred, indicating that the SVA and NL are well-aligned.** We randomly select 50 NL-SVA pairs from our synthetic dataset, and employ human experts to evaluate the accuracy both before and after the selection via bidirectional translation. The results indicate that the accuracy improves from 68% to 96%, showing bidirectional translation can serve as an effective selection method. We also show how bidirectional translation identifies subtle errors in Figure 2. The original SVA can cheat both formal verification and LLM-as-a-judge, but it cannot pass the equivalence check after bidirectional translation.

Based on these observations, we propose an efficient data synthesis framework for NL2SVA tasks. Specifically, our process begins with a collection of open-source RTL designs. For each design, we

A Case Study of Bidirectional Data Selection

Original SVA:

```
asrt_term_complement: assert property (
  @(posedge i_clk) disable iff (tb_reset)
  ctrl_comp |-> (term == (~mux_out + 1))
  and !ctrl_comp |-> (term == mux_out)
);
```

Formal Verification: **Passed.**

NL translated from SVA:

The term value must be the two's complement of mux_out when ctrl_comp is high, and equal to mux_out when ctrl_comp is low.

LLM-as-a-judge: **Passed.**

New SVA re-translated from NL:

```
asrt_term_complement: assert property (
  @(posedge i_clk) disable iff (tb_reset)
  ( ctrl_comp |-> (term == (~mux_out + 1)) )
  and ( !ctrl_comp |-> (term == mux_out) )
);
```

Equivalent Checking: **Failed.**

Remark: In SVA syntax, the **and** operator has higher precedence than the implication ($| \rightarrow$), which makes the original SVA a meaningless tautology that can pass formal verification under any RTL code. Due to the limited knowledge, this subtle error cannot be identified by LLMs. However, with bidirectional translation, we regenerate a new SVA that is not logically equivalent to the original one, thereby detecting this erroneous data.

Figure 2: An example of bidirectional data selection.

first employ a general-purpose LLM to generate multiple NL verification properties and their SVA counterparts. They are subsequently filtered by a formal verification tool, resulting in a large-scale, high-quality seed SVA dataset. Next, we perform bidirectional translation to further align the NL-SVA pairs. Each SVA is first translated into NL and then back into SVA by the LLM. Formal verification tools are then used to check the logical equivalence between the original and re-translated SVAs. Only the equivalent SVAs and their corresponding NLs are retained. Finally, with further difficulty and LLM-as-a-judge selection, followed by reasoning enhancement, we obtain a high-quality NL2SVA dataset. The overview of our data synthesis framework is shown in Figure 3.

To show the effectiveness of our data synthesis method, we develop CodeV-SVA, a series of fine-tuned LLMs for NL2SVA. Specifically, with the synthesized data, we perform supervised fine-tuning (SFT) on open-source models, including Qwen3 8B / 14B [27]. We evaluate them on the mainstream NL2SVA benchmark, FVEval-NL2SVA [15]. The results show that CodeV-SVA-14B achieves 75.8% on NL2SVA-Human and 84.0% on NL2SVA-Machine in Func.@1, matching or surpassing SOTA general-purpose LLMs such as GPT-5 and DeepSeek-R1, while requiring minimal deployment resources. We plan to open-source our dataset, models, and training pipeline.

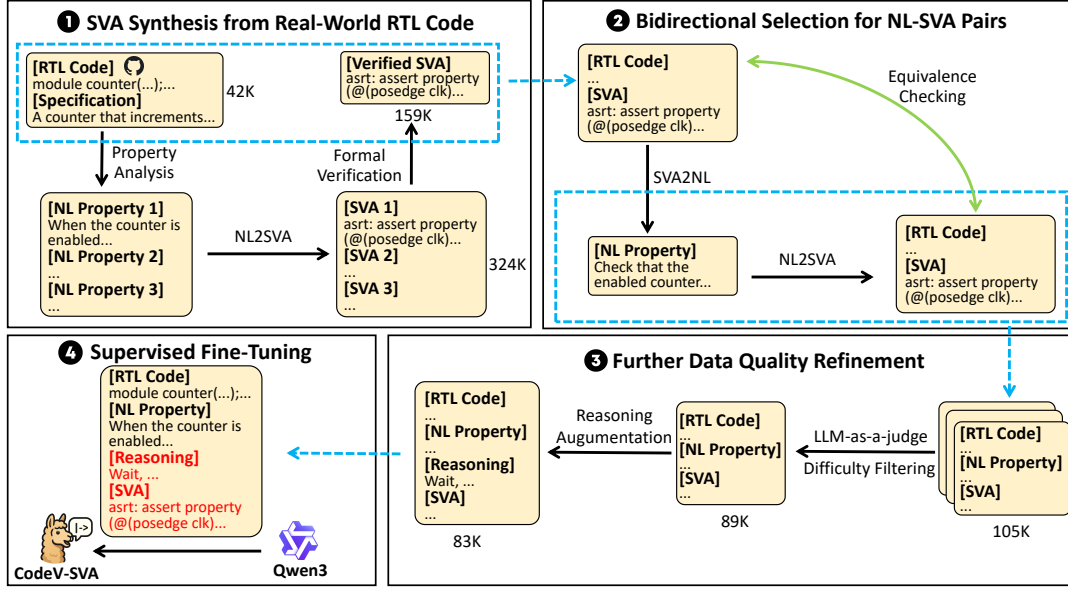


Figure 3: The overview of our data synthesis and training framework.

2 Related Work

Using LLMs to assist hardware verification is a promising application of LLMs for EDA [39], encompassing LLM-based testbench generation [24, 25], LLM-assisted RTL bug localization and repair [4, 18], LLM-aided Universal Verification Methodology (UVM) [13, 34], and LLM for SVA generation [5, 15, 21, 32, 33], which is the primary focus of this paper.

Most LLM-based SVA generation approaches employ existing general-purpose LLMs to generate natural-language properties and SVAs: AssertLLM [33] uses the Retrieval Augmented Generation (RAG) to improve SVA generation flow; AssertionForge [5] constructs Knowledge Graphs (KGs) to help LLMs better understand the relationships between specifications and RTL; DeepAssert [29] guides LLMs to generate SVAs by analyzing the invocation relationships between modules. In contrast, we use synthesized data to train specialized LLMs rather than employ a general model, thereby further improving the accuracy and efficiency of SVA generation.

On the other hand, VERT [21] collects SVAs and RTL code from open-source repositories to finetune LLMs for generating syntactically correct and verifiable SVAs. However, its training data lacks NL properties, making the LLM unable to generate SVAs from NL intents directly provided by human engineers. Hybrid-NL2SVA [32] collects 4K SVAs from textbooks and uses LLMs to generate NL explanations as paired training data, but the small size of the training data limits its effectiveness. We expand the dataset to 83K with RTL-grounded synthesis and improve data quality by bidirectional selection. Moreover, since Hybrid-NL2SVA’s datasets are not publicly available, we are unable to make a performance comparison.

3 Methods

Preliminaries. We follow the problem definition and evaluation protocol for the NL2SVA translation task in FVEval [15]. Specifically, we take RTL code c and a natural-language verification property x

as the input of an LLM $\mathcal{M}$, and then use a formal verification tool (e.g., JasperGold) to check the logical equivalence (denoted as $\sim$) between the model-generated SVA $\mathcal{M}(c, x)$ and the ground-truth SVA $\hat{y}$ given by human experts. Formally, the NL2SVA translation of x by $\mathcal{M}$ is **functionally correct** if and only if $\mathcal{M}(c, x) \sim \hat{y}$.

Overview. This section presents the data synthesis and training framework of CodeV-SVA in detail, which consists of 4 stages: **1 SVA Synthesis from Real-World RTL Code.** To alleviate the scarcity of SVA data, we use a general-purpose LLM to analyze a large scale of open-source RTL code and generate corresponding NL properties and SVAs, followed by filtering out verifiable SVAs by a formal verification tool. **2 Bidirectional Selection for NL-SVA Pairs.** Since a verifiable SVA may not fully capture the NL semantics (e.g., `assert property (1'b1)`), we align NL-SVA via bidirectional translation: SVAs are back-translated to NLs and then re-translated into new SVAs. Only the instances whose newly generated SVAs are proven equivalent to the original one by formal tools are retained. **3 Further Data Quality Refinement.** We integrate human-expert insights into an LLM-as-a-judge selection and remove trivial data pairs by a weaker general-purpose LLM to enhance training effectiveness. Then, we use a strong reasoning LLM, DeepSeek-R1, to augment the dataset with reasoning trajectories. **4 Supervised Fine-Tuning.** With the synthetic NL2SVA dataset, we fine-tune general-purpose LLMs to obtain CodeV-SVA.

3.1 SVA Synthesis from Real-World RTL Code

(1) Open Source RTL Code Curation. Unlike SVA data, open-source RTL code is sufficiently abundant. We choose the CodeV dataset [37] for SVA synthesis, which contains 165K RTL code (denoted as $\{c_i\}$) collected from GitHub and corresponding specification (denoted as $\{s_i\}$) generated by LLMs. To collect the code more suitable for verification scenarios, we use Yosys [31] to analyze the signals in each RTL and filter out 42K instances $\{(c_i, s_i)\}$ with clock

and reset signals, as they typically involve temporal constraints and can thus inspire the model to generate more complex SVAs.

(a) The Prompt and an Example of Property Analysis

You are given a hardware specification together with its Verilog implementation. You need to provide a list of verification properties for this design.

Example:

Specification

...the program counter in Verilog that increments its address by 1 every clock cycle when the enable signal is high.

Code Implementation

```
...
always @(posedge clock or negedge rst) begin
    ...
    else begin
        if(en) pc_addr <= pc_addr+1;
        else pc_addr <= pc_addr;
    end
end
...
```

[Response]

Property 1: When disabled, the counter must hold its value.

(b) The Prompt and an Example of SVA2NL

You are given a hardware design in Verilog and a SVA that verifies some property of the design. Your task is to describe the properties being checked by the SVA in natural language.

Example:

Hardware Design

```
...
always @(posedge clk or posedge reset) begin
    ...
    if (cmd_valid == 1) begin
        cmd_reg = cmd;
        busy <= 1;
    end
end
...
```

SVA

```
asrt: assert property (
    @(posedge clk) disable iff (tb_reset)
    (cmd_valid && !busy) | => busy
);
```

[Response]

When a command is issued while the controller is idle, the controller must become busy in the next cycle.

Figure 4: Concise prompts and examples of NL verification property analysis (Sec. 3.1) and SVA2NL (Sec. 3.2).

(2) Natural-Language Verification Property Analysis. To improve the diversity and accuracy of the synthesis process, we first employ DeepSeek-V3.1 to decompose the specification $\{s_i\}$ into multiple independent properties $\{x_{ij}\}$ (see Figure 4a for prompts and example), helping the model better focus on each subtask. There are 324K generated properties in total.

(3) SVA Generation and Verification. We again feed each synthetic NL property x_{ij} with its corresponding RTL code c_i into DeepSeek-V3.1, which generates an SVA y_{ij} that can be used to check whether x_{ij} holds. To perform an initial screening of high-quality SVAs, we use JasperGold to identify the formally verified SVAs $\{y_{ij}^*\}$ under the RTL code c_i , yielding our original SVA corpora $C = \{(\{y_{ij}^*\}_{j=1}^{m_i}, c_i)\}$, where m_i is the number of the verified SVAs for c_i . Finally, we obtain 159K SVA instances for the subsequent synthesis of NL-SVA paired data.

3.2 Bidirectional Selection for NL-SVA Pairs

Although SVA y_{ij}^* passes formal verification, it does not imply that it fully reflects the semantics of NL x_{ij} , since the RTL code can also satisfy weaker SVAs, and training on such data would significantly degrade the model’s performance (see Table 4). We first attempt to include SVA tutorials [2] in the prompt to assist the model, but the performance shows no considerable improvement (see Table 1), so we choose to conduct data selection.

Table 1: The performance (%) of DeepSeek-R1 with and without tutorial prompts. See Sec. 4.2 for details of benchmarks.

Method	NL2SVA-Human		NL2SVA-Machine	
	Func.@1	Func.@16	Func.@1	Func.@16
w/o Tutorial	74.6	90.3	81.0	93.3
w/ Tutorial	71.5	90.0	83.7	93.7

Our main selection method is bidirectional translation (see Sec. 1), which proceeds in two steps:

(1) SVA-to-NL-to-SVA Translation. The SVA y_{ij}^* is first translated to NL property x_{ij}^* by DeepSeek-V3.1, and then translated back to another SVA y'_{ij} . For SVA-to-NL, we use few-shot examples (Figure 4b) to guide the model to generate high-level NL properties rather than describe signal relationships directly. For NL-to-SVA, we incorporate the signals extracted from the original SVA into the prompt as hints to reduce the uncertainty of LLM outputs.

(2) Data Filtering Based on Formal Equivalence Checking. Since the misalignment may arise in any of the two translation directions, if the regenerated SVA is logically equivalent to the original, the corresponding NL-SVA pair is likely to be consistent. Following FVEval [15], we use JasperGold to formally verify the logical equivalence between two SVAs. More formally, we construct the aligned dataset $\mathcal{D}$ as follow:

$$\mathcal{D} = \{(c_i, x_{ij}^*, y'_{ij}) \mid x_{ij}^* = \mathcal{M}^{-1}(c_i, y_{ij}^*), y'_{ij} = \mathcal{M}(c_i, x_{ij}^*), y_{ij}^* \sim y'_{ij}\},$$

where $\mathcal{M}$ and $\mathcal{M}^{-1}$ is the LLM’s NL-to-SVA and SVA-to-NL process, and $(\cdot \sim \cdot)$ is the equivalence verification. After filtering, we obtain 105K NL-SVA pairs with corresponding RTL code.

3.3 Further Data Quality Refinement

Besides bidirectional data selection, we apply additional general techniques to further refine the data quality, including:

(1) LLM-as-a-judge integrated with expert priors. We employ human experts to analyze LLM errors that bidirectional translation could not identify, and categorize the errors into 4 types:

logical misalignment, signal inconsistency, RTL misunderstanding, and mapping the NL property to the wrong SVA object. We ask DeepSeek-V3.1 to select the NL-SVA pairs without these errors.

(2) **Difficulty Filtering via a Weaker LLM.** To improve training efficiency, we use Qwen3-8B, a general-purpose LLM with weak SVA generation ability (see Table 2), to generate 5 SVAs $\{y_{ijk}\}_{k=1}^5$ for each NL x_{ij}^* and remove the instance where all SVAs are equivalent to y'_{ij} , thereby filtering trivial data points.

(3) **Reasoning Trajectory Augmentation.** Following OpenAI o1 [22], to fully leverage the model’s reasoning capability during NL2SVA, we use a reasoning-enhanced LLM to augment the dataset with long reasoning. Specifically, we feed each NL x_{ij}^* to DeepSeek-R1 and obtain the answer SVA y'_{ij} with reasoning trajectory r_{ij} . We retain only the data whose $y'_{ij} \sim y'_{ij}$, since the correct final answer often implies a correct reasoning trajectory [35].

3.4 Supervised Fine-Tuning

Finally, we obtain an NL2SVA training dataset of 83K instances (i.e., $\{(c_i, x_{ij}^*, r_{ij}, y'_{ij})\}$). To evaluate the quality of the dataset, we fine-tune general-purpose LLMs on it. We treat the RTL code c_i as DUT, and concatenate it with the NL x_{ij}^* as the input of the model. For training label $\tilde{y}_{ij}$, we follow the DeepSeek-R1 format to place the reasoning trajectory between special tokens, followed by the SVA answer, i.e., $\tilde{y}_{ij} = \langle \text{think} \rangle r_{ij} \langle / \text{think} \rangle y'_{ij}$. Formally, we minimize the following objective loss:

$$L_{\text{SFT}}(\theta) = - \sum_{i=1}^N \sum_{j=1}^{M_i} \sum_{t=1}^{T_{ij}} \log P(\tilde{y}_{ij}^{(t)} | \tilde{y}_{ij}^{(<t)}, c_i || x_{ij}^*; \theta),$$

where N denotes the number of RTL codes, M_i denotes the number of training data associated with the i -th RTL, T_{ij} denotes the number of tokens in $\tilde{y}_{ij}$, $\tilde{y}_{ij}^{(t)}$ and $\tilde{y}_{ij}^{(<t)}$ denote the t -th token of $\tilde{y}_{ij}$ and its preceding tokens, and θ denotes the model parameters. We optimize the log probability of the output tokens with respect to the training labels, thereby improving the model’s NL2SVA performance.

4 Experiments

In this section, we present our implementation details (Sec. 4.1). We conduct a series of experiments (settings in Sec. 4.2) to compare the performance between CodeV-SVA and other LLMs (Sec. 4.3), the contributions of each component in our method (Sec. 4.4) and the role of CodeV-SVA in an end-to-end verification workflow (Sec. 4.5).

4.1 Implementation Details

In data synthesis, we set the temperature to 0.8 for NL property generation and SVA2NL translation to increase diversity. We use greedy decoding for NL2SVA to ensure the accuracy of the generated SVAs. In SFT, we use the LlamaFactory [38] framework to fine-tune Qwen3 8B / 14B. The models are trained for 2 epochs with a learning rate of $2e-5$ and a batch size of 128. The context length is 32,768. The training of 8B and 14B models takes 8 and 12 hours on 8 H800-80G GPUs. We use JasperGold 2023.12 for evaluation.

4.2 Experimental Settings

Benchmarks. To show the performance of CodeV-SVA, we evaluate it on NL2SVA-Human and NL2SVA-Machine of FVEval [15]. NL2SVA-Human covers expert-written ground-truth SVAs and NL descriptions. NL2SVA-Machine randomly generates a batch of SVAs by manual rules, then uses LLMs to generate the NL descriptions. We employ human experts to recheck the benchmarks, correcting or removing erroneous tests. We perform 13-gram decontamination [9] on the training datasets to avoid contamination of benchmarks. **Metrics.** We use full functional correctness in FVEval [15] as the metric, i.e., the model-generated SVA holds if and only if the ground-truth SVA holds. We use the pass@ k [8] metric to measure whether the model can generate at least one correct SVA within k attempts, denoted as Func.@ k :

$$\text{Func.@}k := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right],$$

where n is the number of samples for a problem and c is the number of correct SVAs generated by LLMs. In this work, we set $n = 32$.

Baselines. We compare CodeV-SVA with advanced general-purpose LLMs, including DeepSeek-R1 (2025-05-28) [6], GPT-5 (2025-08-07) [3], DeepSeek-V3.1 (2025-08-21) [7] and GPT-4o (2024-11-20) [1]. We also evaluate the specialized RTL generation LLMs, including RTLCoder-Deepseek-v1.1 [17] and CodeV-R1-Qwen-7B [40], as their RTL generation capabilities may transfer to SVA generation. We also evaluate Qwen3-8B and 14B [27] to show the performance improvement from training data. All models are evaluated with a temperature of 0.8 and a top_p of 0.95.

4.3 Main Results

Table 2: The evaluation results (%) of CodeV-SVA and other LLMs. “F@ k ” refers to Func.@ k score (Section 4.2).

Model	NL2SVA-Human			NL2SVA-Machine		
	F@1	F@16	F@32	F@1	F@16	F@32
<i>(Advanced General-Purpose Models)</i>						
DeepSeek-R1-671B	<u>74.6</u>	90.3	<u>90.4</u>	81.0	93.3	94.3
GPT-5	71.8	<u>90.2</u>	92.7	81.8	93.2	94.3
DeepSeek-V3.1-671B	63.1	81.4	84.9	<u>83.8</u>	92.9	93.6
GPT-4o	64.1	75.2	78.1	68.5	81.3	83.7
<i>(Specialized RTL Generation Models)</i>						
RTLCoder-DS-v1.1-6.7B	25.9	58.8	65.8	21.7	54.8	60.8
CodeV-R1-Qwen-7B	25.2	55.8	61.6	37.4	76.6	83.0
<i>(Open-Source Foundation Models)</i>						
Qwen3-8B	32.3	71.6	74.0	46.1	88.0	90.5
Qwen3-14B	61.6	86.1	87.7	75.3	92.7	94.3
<i>(Ours)</i>						
CodeV-SVA-8B	72.0	88.8	<u>90.4</u>	83.5	96.3	97.2
CodeV-SVA-14B	75.8	89.4	<u>90.4</u>	84.0	<u>94.9</u>	<u>95.8</u>

The evaluation results of CodeV-SVA on FVEval-NL2SVA are shown in Table 2. The results demonstrate that CodeV-SVA’s performance leads most general-purpose LLMs:

(1) **Our CodeV-SVA-14B model establishes new state-of-the-art results on the Func.@1 scores of both benchmarks, surpassing general-purpose LLMs as well as specialized RTL generation LLMs, demonstrating the effectiveness of our data synthesis and training pipeline.** Notably, CodeV-SVA-14B surpasses or matches its teacher model DeepSeek-R1-671B and expensive proprietary LLM GPT-5 across all Func.@ k of the two NL2SVA benchmarks, providing a clear computational and cost efficiency advantage. Furthermore, both the 8B and 14B models push beyond previous models’ upper bound on Func.@32 of NL2SVA-Machine.

(2) **Both CodeV-SVA-8B and 14B achieve substantial performance gains over their base models, Qwen3-8B and 14B.** In particular, CodeV-SVA-8B achieves 39.7% and 37.4% improvements over Qwen3-8B in Func.@1 of NL2SVA-Human and NL2SVA-Machine, respectively, demonstrating that **high-quality synthesized data** is the key to improving SVA generation capabilities of open-source general-purpose LLMs.

4.4 Ablation Studies

To investigate the contribution of each component in the data synthesis framework of CodeV-SVA, we conduct a series of ablation studies, including ablations on the sources of SVA in the training dataset (Table 3) and on the data refinement and selection methods (Table 4). We use Qwen3-8B as the base model in all ablations.

(1) **LLM-synthesized SVAs are high-quality training labels, which can contribute to better performance than collecting or rewriting existing SVAs from open-source repositories.** We compare the SVAs generated by LLMs (Section 3.1) with the SVAs collected or rule-based rewritten from open-source repositories in prior work, including DeepCircuitX [16] and VERT [21]. Specifically, we use DeepSeek-V3.1 to perform SVA2NL (Section 3.2) on the open-source SVAs $\{y_i\}$ with RTL code $\{c_i\}$ to obtain the NL properties $\{x_i\}$, then train Qwen3-8B on $\{(c_i, x_i, y_i)\}$. We compare the resulting LLMs with LLMs trained on the same amount of synthetic data of our method (random 5K). The results (Table 3) show that training LLMs on open-source SVAs leads to a significant degradation in performance, which is likely due to the variable data quality across these repositories, which makes training unstable.

Table 3: The performance (%) comparison between different SVA sources in training data. Synthesized: LLM-synthesized SVAs. DeepCircuitX: SVAs in open-source repositories. VERT: Rewriting open-source SVAs with manual rules.

SVA Source	Data Size	NL2SVA-Human			NL2SVA-Machine		
		F@1	F@16	F@32	F@1	F@16	F@32
Synthesized (ours)	5K	55.4	84.0	86.3	75.7	94.0	95.1
DeepCircuitX	5K	22.3	59.0	64.4	39.2	58.8	61.1
VERT	20K	1.9	8.8	11.0	6.4	16.0	19.1

(2) **In data refinement (Section 3.3), reasoning augmentation provides a clear performance boost for the model. Difficulty filtering and LLM-as-a-judge serve complementary roles that reduce data size and improve training efficiency.** In Table 4, removing the reasoning trajectories (line 2), the performance drops across all metrics on both benchmarks, highlighting

Table 4: The performance (%) of removing each component in data refinement (Sec. 3.3) and selection (Sec. 3.2 and 3.1). R: Reasoning trajectories; D: Difficulty filtering; J: LLM-as-a-judge; B: Bidirectional data selection; V: Formal verification under RTL code.

Method	Data Size	NL2SVA-Human			NL2SVA-Machine		
		F@1	F@16	F@32	F@1	F@16	F@32
CodeV-SVA (ours)	83K	72.0	88.8	90.4	83.5	96.3	97.2
<i>(Ablation on data refinement)</i>							
w/o R	89K	63.9	80.0	82.2	81.1	91.2	92.2
w/o R, D	97K	62.2	80.4	82.2	81.1	91.6	92.6
w/o R, D, J	105K	63.5	80.1	82.2	77.2	91.5	92.6
<i>(Ablation on data selection)</i>							
w/o R, D, J, B	159K	51.2	76.4	80.8	76.5	92.1	92.9
w/o R, D, J, B, V	324K	44.1	73.5	77.4	78.8	93.0	94.0

the role of the reasoning paradigm in the NL2SVA task. Using difficulty filtering (line 3) and LLM-as-a-judge (line 4), we remove 15% low-quality samples and achieve a modest improvement in the model’s Func.@1 score.

(3) **Bidirectional data selection (Section 3.2) yields the most substantial performance gain for NL2SVA-Human (12.3% in Func.@1, line 4 vs. line 5) among all components of our data synthesis framework, fully demonstrating the method’s effectiveness in enhancing data quality.** Furthermore, by reducing the amount of data by 34%, the bidirectional selection method greatly improves training efficiency.

(4) **With only SVAs that pass formal verification under RTL code (Section 3.1), the model achieves superior performance with less than half of the training data (line 5 vs. line 6).** Although there is a slight performance drop on NL2SVA-Machine, which may be attributed to the distribution gap between the rule-based synthesized benchmark and real-world SVAs, it is easy to recover via subsequent data refinement. The whole ablation studies show that the selection stages in our method continuously improve the performance while reducing the data size.

4.5 CodeV-SVA in End-to-End Verification

We evaluate the effectiveness of CodeV-SVA within a fully automated end-to-end verification framework: given the specification document of a hardware design as input, LLMs autonomously analyze the design constraints and generate the corresponding SVAs. Specifically, we modify the AssertionForge framework [5] by dividing it into two components: (1) Spec2NL: General-purpose LLMs (e.g., GPT-4o) are used to analyze the specifications and RTL code, generating NL properties; (2) NL2SVA: The NL properties are translated into corresponding SVAs by CodeV-SVA. We keep the same framework in AssertionForge, modify its prompts for our model, and evaluate it on 5 RTL designs [23, 33] in the original paper. We keep GPT-4o and DeepSeek-R1 in the Spec2NL task, while we choose CodeV-SVA for the NL2SVA task. Then we count the total number of generated SVAs (#SVA), the number of syntax correct

Table 5: Evaluation results of CodeV-SVA on AssertionForge.

Spec2NL	NL2SVA	#SVA	#SynC	#Proven
APB				
GPT-4o	GPT-4o	436	377	106
	CodeV-SVA-8B (ours)	442	384	180
DeepSeek-R1	DeepSeek-R1	515	281	122
	CodeV-SVA-8B (ours)	529	466	211
ETHMAC				
GPT-4o	GPT-4o	720	565	49
	CodeV-SVA-8B (ours)	721	579	76
DeepSeek-R1	DeepSeek-R1	665	371	75
	CodeV-SVA-8B (ours)	660	532	84
OPENMSP430				
GPT-4o	GPT-4o	1013	777	196
	CodeV-SVA-8B (ours)	1044	859	484
DeepSeek-R1	DeepSeek-R1	1222	496	144
	CodeV-SVA-8B (ours)	1215	917	502
SOCKIT				
GPT-4o	GPT-4o	251	173	31
	CodeV-SVA-8B (ours)	251	202	49
DeepSeek-R1	DeepSeek-R1	232	122	40
	CodeV-SVA-8B (ours)	247	178	58
UART				
GPT-4o	GPT-4o	265	186	54
	CodeV-SVA-8B (ours)	270	196	73
DeepSeek-R1	DeepSeek-R1	258	91	40
	CodeV-SVA-8B (ours)	266	185	70

SVAs (#Sync), and the number of SVAs that pass formal verification (#Proven) as metrics (COI coverage in AssertionForge’s paper is already saturated; thus, CodeV-SVA only brings improvement to OPENMSP430, and the remaining designs achieve nearly 100% coverage).

Given the same NL plans, CodeV-SVA generates significantly more syntax correct and verifiable SVAs during the NL2SVA stage (Table 5). Notably, on OPENMSP430, a complex design with a 129-page specification document and 29 RTL files, CodeV-SVA-8B generates far more verifiable SVAs than GPT-4o (2.5×) and DeepSeek-R1 (3.5×), demonstrating its advantage over general-purpose LLMs in an end-to-end verification framework.

5 Conclusion

In this paper, we propose CodeV-SVA, a data synthesis and model training pipeline for SVA generation. This pipeline alleviates the scarcity of high-quality SVA data by synthesizing SVAs from real-world RTL code, and further improves data quality by filtering model-generated NL-SVA data pairs using a bidirectional translation method, followed by data refinement. We train CodeV-SVA-8B and 14B with this pipeline and evaluate them on NL2SVA-Human and NL2SVA-Machine benchmarks of FVEval. Experimental results show that CodeV-SVA significantly outperforms general-purpose LLMs on NL2SVA tasks, demonstrating the effectiveness of our data synthesis and training pipeline.

6 Acknowledgements

This work is partially supported by the NSF of China (Grants No.62402477, 62341411), Strategic Priority Research Program of the Chinese Academy of Sciences (Grants No.XDB0660300, XDB0660301, XDB0660302, XDB0660200, XDB0660201, XDB0660202), and Youth Innovation Promotion Association CAS.

References

- [1] OpenAI 2024. *GPT-4o System Card*. OpenAI. <https://openai.com/index/gpt-4o-system-card/>. Accessed: 2025-11-15.
- [2] SystemVerilog .io 2024. *SystemVerilog Assertions Basics*. SystemVerilog .io. <https://www.systemverilog.io/verification/sva-basics/>. Accessed: 2025-11-15.
- [3] OpenAI 2025. *GPT-5 System Card*. OpenAI. <https://openai.com/index/gpt-5-system-card/>. Accessed: 2025-11-15.
- [4] Yunsheng Bai, Ghaith Bany Hamad, Chia-Tung Ho, Syed Suhaib, and Haoxing Ren. 2025. FVDebug: An LLM-Driven Debugging Assistant for Automated Root Cause Analysis of Formal Verification Failures. *arXiv:2510.15906* [cs.AR] <https://arxiv.org/abs/2510.15906>
- [5] Yunsheng Bai, Ghaith Bany Hamad, Syed Suhaib, and Haoxing Ren. 2025. AssertionForge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL. *arXiv:2503.19174* [cs.AI] <https://arxiv.org/abs/2503.19174>
- [6] DeepSeek-AI 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL] <https://arxiv.org/abs/2501.12948>
- [7] DeepSeek-AI 2025. DeepSeek-V3 Technical Report. *arXiv:2412.19437* [cs.CL] <https://arxiv.org/abs/2412.19437>
- [8] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374* [cs.LG] <https://arxiv.org/abs/2107.03374>
- [9] Tom B. Brown et al. 2020. Language Models are Few-Shot Learners. *arXiv:2005.14165* [cs.CL] <https://arxiv.org/abs/2005.14165>
- [10] Samuele Germiniani and Graziano Pravadelli. 2022. HARM: A Hint-Based Assertion Miner. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4277–4288. doi:10.1109/TCAD.2022.3197525
- [11] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. A Survey on LLM-as-a-Judge. *arXiv:2411.15594* [cs.CL] <https://arxiv.org/abs/2411.15594>
- [12] Mohammad Reza Heidari Iman, Gert Jervan, and Tara Ghasempouri. 2024. ARTmine: Automatic Association Rule Mining with Temporal Behavior for Hardware Verification. In *2024 Design, Automation and Test in Europe Conference and Exhibition (DATE)*. 1–6. doi:10.23919/DATE58400.2024.10546742
- [13] Yuchen Hu, Junhao Ye, Ke Xu, Jialin Sun, Shiyue Zhang, Xinyao Jiao, Dingrong Pan, Jie Zhou, Ning Wang, Weiwei Shan, Xinwei Fang, Xi Wang, Nan Guan, and Zhe Jiang. 2024. UVLLM: An Automated Universal RTL Verification Framework using LLMs. *arXiv:2411.16238* [cs.AR] <https://arxiv.org/abs/2411.16238>
- [14] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, and Jeyavijayan Rajendran. 2024. (Security) Assertions by Large Language Models. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4374–4389. doi:10.1109/TIFS.2024.3372809
- [15] Minwoo Kang, Mingjie Liu, Ghaith Bany Hamad, Syed M. Suhaib, and Haoxing Ren. 2025. FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware. In *2025 Design, Automation and Test in Europe Conference (DATE)*. 1–6. doi:10.23919/DATE64628.2025.10992720
- [16] Zeju Li, Changran Xu, Zhengyuan Shi, Zedong Peng, Yi Liu, Yunhao Zhou, Lingfeng Zhou, Chengyu Ma, Jianyuan Zhong, Xi Wang, Jieru Zhao, Zhufei Chu, Xiaoyan Yang, and Qiang Xu. 2025. DeepCircuitX: A Comprehensive Repository-Level Dataset for RTL Code Understanding, Generation, and PPA Analysis. *arXiv:2502.18297* [cs.LG] <https://arxiv.org/abs/2502.18297>
- [17] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2025. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *Trans. Comp.-Aided Des. Integr. Cir. Sys.* 44, 4 (April 2025), 1448–1461. doi:10.1109/TCAD.2024.3483089
- [18] Hongqin Lyu, Yunlin Du, Yonghao Wang, Zhiteng Chao, Tiancheng Wang, and Huawei Li. 2025. AssertFix: Empowering Automated Assertion Fix via Large Language Models. *arXiv:2509.23972* [cs.AR] <https://arxiv.org/abs/2509.23972>
- [19] Hongqin Lyu, Yonghao Wang, Yunlin Du, Mingyu Shi, Zhiteng Chao, Wenxing Li, Tiancheng Wang, and Huawei Li. 2025. AssertGen: Enhancement of LLM-aided Assertion Generation through Cross-Layer Signal Bridging. *arXiv:2509.23674* [cs.AR] <https://arxiv.org/abs/2509.23674>
- [20] Ashok B Mehta. 2020. *SystemVerilog Assertions and Functional Coverage*. Springer.
- [21] Anand Menon, Samit S Miftah, Shamik Kundu, Souvik Kundu, Amisha Srivastava, Arnab Raha, Gabriel Theodor Sonnenschein, Suvadeep Banerjee, Deepak Mathakutty, and Kanad Basu. 2025. Enhancing Large Language Models for Hardware

- Verification: A Novel SystemVerilog Assertion Dataset. arXiv:2503.08923 [cs.LG] <https://arxiv.org/abs/2503.08923>
- [22] OpenAI. 2024. OpenAI o1 System Card. arXiv:2412.16720 [cs.AI] <https://arxiv.org/abs/2412.16720>
- [23] OpenCores. 2024. OpenCores: Open Source Hardware Designs. OpenCores Website. <https://opencores.org/> Accessed: June 28, 2024.
- [24] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2024. AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design. arXiv:2407.03891 [cs.SE] <https://arxiv.org/abs/2407.03891>
- [25] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2024. CorrectBench: Automatic Testbench Generation with Functional Self-Correction using LLMs for HDL Design. arXiv:2411.08510 [cs.SE] <https://arxiv.org/abs/2411.08510>
- [26] Karen Sparck Jones. 1988. *A statistical interpretation of term specificity and its application in retrieval*. Taylor Graham Publishing, GBR, 132–142.
- [27] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [28] Enyuan Tian, Yiwei Ci, Qiusong Yang, Yufeng Li, and Zhichao Lyu. 2025. AssertCoder: LLM-Based Assertion Generation via Multimodal Specification Extraction. arXiv:2507.10338 [cs.SE] <https://arxiv.org/abs/2507.10338>
- [29] Yonghao Wang, Jiaxin Zhou, Hongqin Lyu, Zhiteng Chao, Tiancheng Wang, and Huawei Li. 2025. DeepAssert: An LLM-Aided Verification Framework with Fine-Grained Assertion Generation for Modules with Extracted Module Specifications. arXiv:2509.14668 [cs.AR] <https://arxiv.org/abs/2509.14668>
- [30] Hasini Witharana, Yangdi Lyu, Subodha Charles, and Prabhat Mishra. 2022. A Survey on Assertion-based Hardware Verification. *ACM Comput. Surv.* 54, 11s, Article 225 (Sept. 2022), 33 pages. doi:10.1145/3510578
- [31] Clifford Wolf, Johann Glaser, and Johannes Kepler. 2013. Yosys-A Free Verilog Synthesis Suite. In *clifford.fm*. <https://api.semanticscholar.org/CorpusID:202611483>
- [32] Weihua Xiao, Derek Ekberg, Siddharth Garg, and Ramesh Karri. 2025. Hybrid-NL2SVA: Integrating RAG and Finetuning for LLM-based NL2SVA. In *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*. 1–10. doi:10.1109/MLCAD65511.2025.11189208
- [33] Zhiyuan Yan, Wenji Fang, Mengming Li, Min Li, Shang Liu, Zhiyao Xie, and Hongce Zhang. 2025. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (Tokyo, Japan) (ASP-DAC '25)*. Association for Computing Machinery, New York, NY, USA, 614–621. doi:10.1145/3658617.3697756
- [34] Junhao Ye, Yuchen Hu, Ke Xu, Dingrong Pan, Qichun Chen, Jie Zhou, Shuai Zhao, Xinwei Fang, Xi Wang, Nan Guan, and Zhe Jiang. 2025. From Concept to Practice: an Automated LLM-aided UVM Machine for RTL Verification. arXiv:2504.19959 [cs.AR] <https://arxiv.org/abs/2504.19959>
- [35] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465 [cs.LG] <https://arxiv.org/abs/2203.14465>
- [36] Bolin Zhang, Jiahao Wang, Qianlong Du, Jiajun Zhang, Zhiying Tu, and Dianhui Chu. 2025. A Survey on Data Selection for LLM Instruction Tuning. *Journal of Artificial Intelligence Research* 83 (Aug. 2025). doi:10.1613/jair.1.17625
- [37] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yanan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, and Xing Hu. 2025. CodeV: Empowering LLMs with HDL Generation through Multi-Level Summarization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025), 1–1. doi:10.1109/TCAD.2025.3604320
- [38] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. arXiv:2403.13372 [cs.CL] <https://arxiv.org/abs/2403.13372>
- [39] Ruizhe Zhong, Xingbo Du, Shixiong Kai, Zhentao Tang, Siyuan Xu, Hui-Ling Zhen, Jianye Hao, Qiang Xu, Mingxuan Yuan, and Junchi Yan. 2023. LLM4EDA: Emerging Progress in Large Language Models for Electronic Design Automation. arXiv:2401.12224 [cs.AR] <https://arxiv.org/abs/2401.12224>
- [40] Yaoyu Zhu, Di Huang, Hanqi Lyu, Xiaoyun Zhang, Chongxiao Li, Wenxuan Shi, Yutong Wu, Jianan Mu, Jinghua Wang, Yang Zhao, Pengwei Jin, Shuyao Cheng, Shengwen Liang, Xishan Zhang, Rui Zhang, Zidong Du, Qi Guo, Xing Hu, and Yunji Chen. 2025. QiMeng-CodeV-R1: Reasoning-Enhanced Verilog Generation. arXiv:2505.24183 [cs.LG] <https://arxiv.org/abs/2505.24183>